

WalletWatch

Résumé de projet

Geoffrey Carpentier

Titre professionnel Développeur Web et Web Mobile

Session de Marseille, septembre 2026

| Le besoin de départ

J'ai réalisé **WalletWatch** pendant ma formation pour réunir le suivi de plusieurs types d'actifs : cryptomonnaies, devises, métaux précieux et une sélection d'actions. Ce projet individuel, sans client ni commanditaire, m'a conduit à définir le besoin, concevoir les écrans, développer l'application et préparer sa mise en service.

Quand un patrimoine est réparti entre plusieurs plateformes, il faut rapprocher les informations pour connaître sa valeur totale. Un tableur peut le faire, mais il demande de préparer les calculs, de mettre les cours à jour et de vérifier chaque saisie. Mon objectif était d'automatiser ce suivi et de rendre les résultats lisibles sur téléphone comme sur ordinateur.

| Ce que permet l'application

L'utilisateur crée un compte, ajoute les actifs qu'il souhaite suivre et enregistre ses achats, ses ventes ou ses sorties sans vente. Il peut consulter son patrimoine et ses positions, ouvrir leur détail, corriger un mouvement et exporter ses données au format CSV.

Le serveur calcule la quantité détenue, le **prix de revient moyen pondéré**, les plus-values et la répartition du patrimoine. Avant un enregistrement, il renvoie un récapitulatif qui permet de contrôler le résultat. L'utilisateur peut aussi créer des seuils, afficher les montants en euros ou en dollars et masquer les valeurs à l'écran.

Les sept écrans couvrent la connexion, le patrimoine, les positions, le détail d'une position, la saisie d'un mouvement, les seuils et le compte. J'ai également prévu les situations moins favorables : chargement, portefeuille vide, erreur réseau, session expirée ou cours indisponible.

| Comment je l'ai construite

L'interface utilise **React** et **Vite**. L'API repose sur **Node.js** et **Express**. **PostgreSQL** conserve les comptes, les actifs et les mouvements ; **Redis** limite les appels aux fournisseurs de cours et garde le dernier cours connu comme valeur de secours. Docker Compose réunit ces services dans un environnement reproductible.

J'ai séparé le serveur en routes, contrôleurs, services et modèles. Cette organisation m'aide à distinguer les échanges avec le navigateur, les règles de calcul et les requêtes SQL. Les calculs financiers restent côté serveur afin d'éviter deux versions différentes d'une même règle.

| Une difficulté concrète

Le prix de revient comprend les frais d'achat. Dix titres achetés à 100 € avec 5 € de frais, puis dix titres à 120 € avec 5 € de frais donnent un coût total de 2 210 €, soit **110,50 € par titre**. Une vente de cinq titres ne change pas le prix de revient des quinze titres restants.

La difficulté apparaît lorsqu'un mouvement ancien est ajouté, corrigé ou supprimé. Vérifier seulement la quantité actuelle ne suffit pas : une vente pouvait être impossible à sa date, même si la position est positive aujourd'hui. Le service reprend donc les mouvements dans l'ordre chronologique et refuse l'opération dès que la quantité deviendrait négative.

J'ai aussi dû traiter deux demandes reçues en même temps. La lecture et l'écriture s'exécutent maintenant dans une transaction PostgreSQL avec un verrou sur l'actif concerné. Ce problème m'a montré qu'un test avec des données simulées ne suffit pas toujours ; les accès concurrents ont été vérifiés avec une vraie base dédiée.

| Sécurité et vérifications

Les mots de passe sont hachés avec bcrypt. Les routes privées vérifient le jeton, l'état du compte et la propriété des données demandées. Une ressource appartenant à un autre compte répond 404, comme une ressource absente. Les valeurs SQL sont passées séparément du texte des requêtes et les entrées sont contrôlées côté serveur.

Sur le commit de référence, les suites rejouées le 10 septembre 2026 comptent **410 tests serveur** et **359 tests d'interface** réussis. Les **22 tests d'intégration PostgreSQL**, exécutés le 7 septembre, vérifient notamment les accès concurrents. Ces résultats prouvent les cas couverts, pas l'absence de tout défaut.

| Limites et suites possibles

WalletWatch suit les actifs saisis, mais pas les liquidités d'un compte bancaire. Le produit d'une vente ne devient donc pas un solde en euros. Les cours dépendent de services extérieurs et de leurs quotas ; le dernier cours connu peut être ancien et doit rester clairement daté.

La liste d'actions est limitée. L'application ne passe pas d'ordre, ne donne pas de conseil d'investissement et ne calcule pas la fiscalité. L'import de fichiers, les annonces et l'administration sont reportés. Une ouverture à de vrais utilisateurs demanderait notamment HTTPS, un canal sécurisé de récupération de compte, des sauvegardes vérifiées et un suivi d'exploitation.

| Ce que je retiens

Ce projet m'a appris à relier un besoin à une règle, puis à son code, son test et son résultat visible. J'ai aussi appris à distinguer ce qu'un test unitaire peut vérifier de ce qui demande une vraie base, un navigateur ou la simulation d'une panne.

| Code de référence

Branche dev, commit [a8a5e6d sur GitHub](#). Certains identifiants techniques et les anciennes maquettes conservent le nom CapitAll, utilisé avant le renommage du produit.